

ЛАБОРАТОРНА РОБОТА № 6

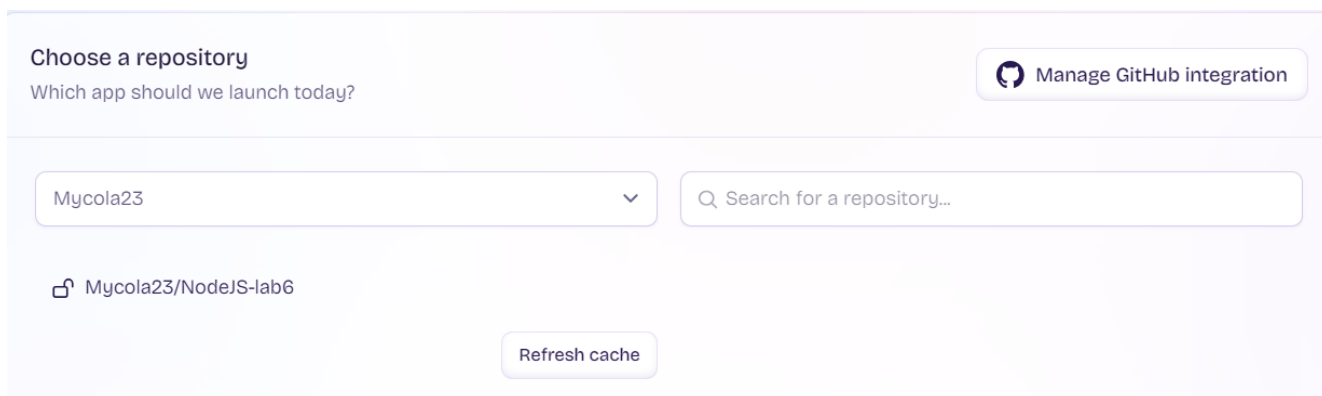
CI/CD та деплой на fly.io

Мета: Метою лабораторної роботи є набуття практичних навичок автоматизації збірки, тестування та розгортання Node.js-застосунку у хмарі шляхом продовження роботи з контейнеризованим REST API з лабораторної роботи №5

Хід роботи:

Завдання 3.

Завдання 3.1. Ручний деплой на fly.io

The screenshot shows the fly.io dashboard. At the top, it says "Choose a repository" and "Which app should we launch today?". There is a button "Manage GitHub integration". Below, there is a dropdown menu showing "Mycola23" and a search bar "Search for a repository...". A link "Mycola23/NodeJS-lab6" is visible. At the bottom, there is a "Refresh cache" button.

Вимоги:

- Зареєструйтесь на fly.io та встановіть flyctl (див. офіційну документацію для вашої ОС). Виконайте автентифікацію.
- Ініціалізуйте застосунок у корені проєкту з лаб. 5. Під час ініціалізації використайте наявний Dockerfile — не створюйте базу даних на fly.io, MongoDB залишається зовнішньою (MongoDB Atlas з лаб. 4).
- Переконайтесь, що згенерований fly.toml коректно посилається на ваш Dockerfile та відкриває потрібний внутрішній порт.
- Задайте секрети застосунку (рядок підключення до MongoDB та інші чутливі значення) через fly secrets.
- Виконайте ручний деплой та переконайтесь, що застосунок доступний за згенерованою адресою: перевірте CRUD-ендпоінти та /health з лаб. 5.
- Ознайомтесь із командами перегляду стану та логів застосунку.

					ДУ «Житомирська політехніка» 26.121.01.006 – Лр6			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Литвиненко М.В			Звіт з лабораторної роботи		Лім.	Арк.
Перевір.		Оринчак А. І.						1
Керівник								8
Н. контр.							ФІКТ Гр. ВТ-24-1[2]	
Зав. каф.								

```

PS D:\diciplens\second-course\NodeJS\lab_06> iwr https://fly.io/install.ps1 | iex
Downloading flyctl...
flyctl was installed successfully to C:\Users\Litvi\.fly\bin\flyctl.exe
Restart your shell, then run 'flyctl --help' to get started
PS D:\diciplens\second-course\NodeJS\lab_06> fly auth login
Opening https://fly.io/app/auth/cli/6d657a6e64626a6567676a7862696371366e676973767a346a7275736b6b616f ...

Waiting for session... Done
successfully logged in as litvinenkomycola263@ukr.net
PS D:\diciplens\second-course\NodeJS\lab_06> fly launch
Scanning source code
Detected a Dockerfile app
Creating app in D:\diciplens\second-course\NodeJS\lab_06

Created app 'lab-06' in organization 'personal'

Admin URL: https://fly.io/apps/lab-06
Hostname: lab-06.fly.dev
Run `fly tokens create deploy -x 999999h` to create a token and set it as the FLY_API_TOKEN secret in your GitHub repository settings
See https://docs.github.com/en/actions/security-guides/using-secrets-in-github-actions

Wrote config file fly.toml
Validating D:\diciplens\second-course\NodeJS\lab_06\fly.toml
✓ Configuration is valid
- building image

Watch your deployment at https://fly.io/apps/lab-06/monitoring

Provisioning ips for lab-06
Dedicated ipv6: 2a09:8280:1::10b:8d0a:0
Shared ipv4: 66.241.125.161
Add a dedicated ipv4 with: fly ips allocate-v4

This deployment will:
* create 2 "app" machines

10 machines in group app, launching a new machine

WARNING The app is not listening on the expected address and will not be reachable by fly-proxy.
You can fix this by configuring your app to listen on the following addresses:
- 0.0.0.0:3000

Found these processes inside the machine with open listening sockets:
PROCESS | ADDRESSES
-----|-----
/.fly/hallpass | [fdaa:70:51c9:a7b:445:ee1f:a040:2]:22

Creating a second machine for high availability and zero downtime deployments.
To disable this, set "min_machines_running = 0" in your fly.toml.

-----
✓ Machine 48e65e5c10e318 [app] was created

PS D:\diciplens\second-course\NodeJS\lab_06> fly secrets set MONGODB_URI="mongodb+srv://vt241lmv_db_user:jpZa90RrUFEQhuPY@m0.bdbnror.mongodb.net/movies_db"
Updating existing machines in 'lab-06' with rolling strategy

-----
✓ [1/2] Machine 48e65e5c10e318 [app] update succeeded
✓ [2/2] Machine d899739be993e8 [app] update succeeded
-----
Checking DNS configuration for lab-06.fly.dev
✓ DNS configuration verified
PS D:\diciplens\second-course\NodeJS\lab_06>

```

		Литвиненко М.В.			ДУ «Житомирська політехніка».26.121.01.006 – Лр6	Арк. 2
		Оринчак А. І.				
Змн.	Арк.	№ докум.	Підпис	Дата		

Рис.1 Встановлення fly.io, ініціалізація застосунку, налаштування secrets

```

{
  "data": [
    {
      "_id": "69d173778b782a51247f3826",
      "title": "The Grand Budapest Hotel",
      "description": "A writer encounters the owner of a high-class hotel, who tells him of his early years.",
      "genre": "Comedy",
      "rating": 4.1,
      "releaseYear": 2014,
      "director": "Wes Anderson",
      "actors": [
        "Ralph Fiennes",
        "Tony Revolori"
      ],
      "createdAt": "2023-10-27T10:15:00.000Z",
      "updatedAt": "2023-10-27T10:15:00.000Z",
      "isClassic": false,
      "id": "69d173778b782a51247f3826"
    }
  ],
  "message": "Movie found"
}

```

```

{
  "status": "UP",
  "database": "connected",
  "timestamp": "2026-04-26T11:37:03.745Z"
}

```

DELETE-method

```

PS D:\dicsiplens\second-course\NodeJS\lab_06> curl.exe -X DELETE https://lab-06.fly.dev/api/movies/69d173778b782a51247f3823
PS D:\dicsiplens\second-course\NodeJS\lab_06> curl.exe https://lab-06.fly.dev/api/movies/69d173778b782a51247f3823
{"message": "Movie not found"}
PS D:\dicsiplens\second-course\NodeJS\lab_06>

```

POST-method

```

PS D:\dicsiplens\second-course\NodeJS\lab_06> curl.exe -X POST https://lab-06.fly.dev/api/movies -H "Content-Type: application/json" -d "@data.json"
{"title": "Interstellarr", "description": "", "genre": "Sci-Fi", "rating": 4.3, "releaseYear": 2015, "director": "Christopher Nollann", "actors": [], "_id": "69edfb339d18e80cc9373264", "createdAt": "2026-04-26T11:46:59.838Z", "updatedAt": "2026-04-26T11:46:59.838Z", "__v": 0, "isClassic": false, "id": "69edfb339d18e80cc9373264"}
PS D:\dicsiplens\second-course\NodeJS\lab_06>

```

Рис.2 успішна перевірка crud та deploy проекту

3.2. Завдання Базовий конвеєр build + test

Налаштуйте конвеєр GitLab CI, який перевіряє код на кожен пуш.

Вимоги:

		Литвиненко М.В.			ДУ «Житомирська політехніка».26.121.01.006 – Лр6	Арк.
		Оринчак А. І.				3
Змн.	Арк.	№ докум.	Підпис	Дата		

- Переконайтесь, що у `package.json` є скрипт `build`, який компілює `TypeScript`.
- Створіть `.gitlab-ci.yml` у корені репозиторію. Визначте стадії `build` та `test` і базовий `Node.js`-образ для задач.
- Задача `build`: встановлення залежностей через `npm ci` та запуск компіляції `TypeScript`.
- Задача `test`: встановлення залежностей та запуск `Jest` із збором покриття коду тестами.
- Додайте кешування `node_modules` між задачами для пришвидшення конвеєра.
- Перевірте роботу конвеєра: запусьте код — конвеєр має бути зеленим. Навмисно внесіть помилку типів або зламайте тест, переконайтесь, що конвеєр впав, після чого відкотіть зміну.

+ 3.3. Завдання Автоматичний деплой зі стадії `deploy`

Розширте конвеєр стадією `deploy`, яка автоматично розгортає застосунок на `fly.io`.

Вимоги:

- Створіть у `fly.io` API-токен з правами деплою та збережіть його в `GitLab` у `Settings` → `CI/CD` → `Variables` як `FLY_API_TOKEN`, увімкнувши налаштування `Protected` та `Masked`.
- Додайте до `.gitlab-ci.yml` стадію `deploy` і відповідну задачу:
 - використовуйте офіційний `Docker`-образ із передвстановленим `flyctl`;
 - викликайте `fly deploy` з прапорцем, що делегує збірку на `fly.io`;
 - обмежте виконання задачі гілкою `main` (або `master`);
- Переконайтесь, що жоден секрет (`FLY_API_TOKEN`, рядок підключення до БД тощо) не потрапив у репозиторій або у логи `CI`.

ci-cd

```
name: CI/CD Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
```

		Литвиненко М.В.			ДУ «Житомирська політехніка».26.121.01.006 – Лр6	Арк.
		Оринчак А. І.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

```

build-and-test:
  runs-on: ubuntu-latest

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: '18'
        cache: 'npm'

    - name: Install dependencies
      run: npm ci

    - name: Build TypeScript
      run: npm run build

    - name: Run Tests with Coverage
      run: npm run test

deploy:
  needs: build-and-test
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  runs-on: ubuntu-latest

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Setup flyctl
      uses: superfly/flyctl-actions/setup-flyctl@master

    - name: Deploy to Fly.io
      run: flyctl deploy --remote-only
      env:
        FLY_API_TOKEN: ${ secrets.FLY_API_TOKEN }

```

		Литвиненко М.В.			ДУ «Житомирська політехніка».26.121.01.006 – Лр6	Арк.
		Оринчак А. І.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

Результат виконання

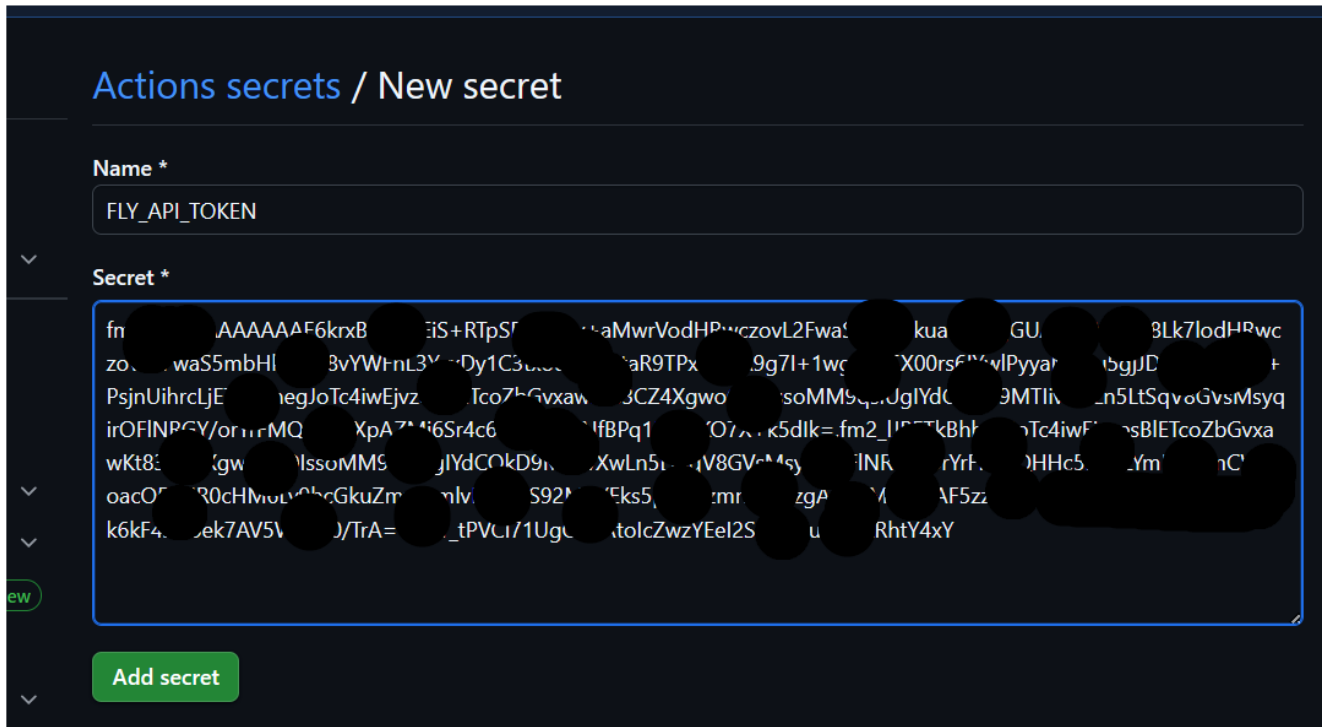


Рис.3 успішна додан секрет токен на GitHub репозиторій

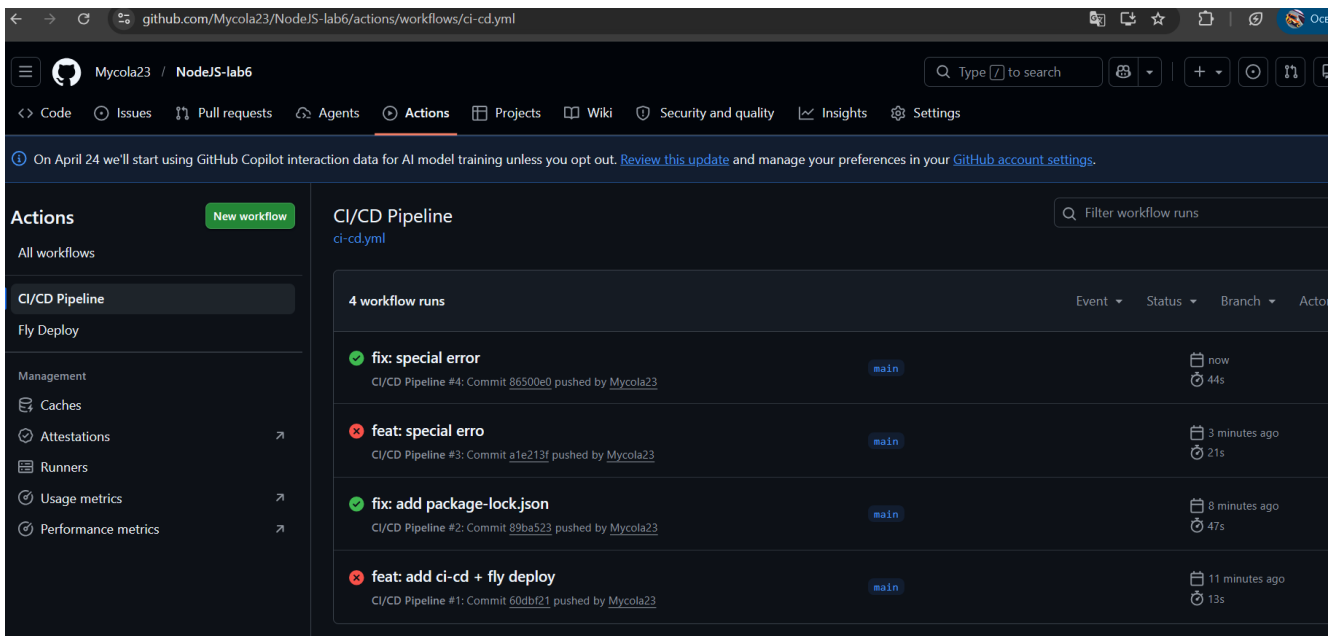


Рис.4 успішно перевірено роботу конвеєру

		Литвиненко М.В.			ДУ «Житомирська політехніка».26.121.01.006 – Лр6	Арк.
		Оринчак А. І.				6
Змн.	Арк.	№ докум.	Підпис	Дата		

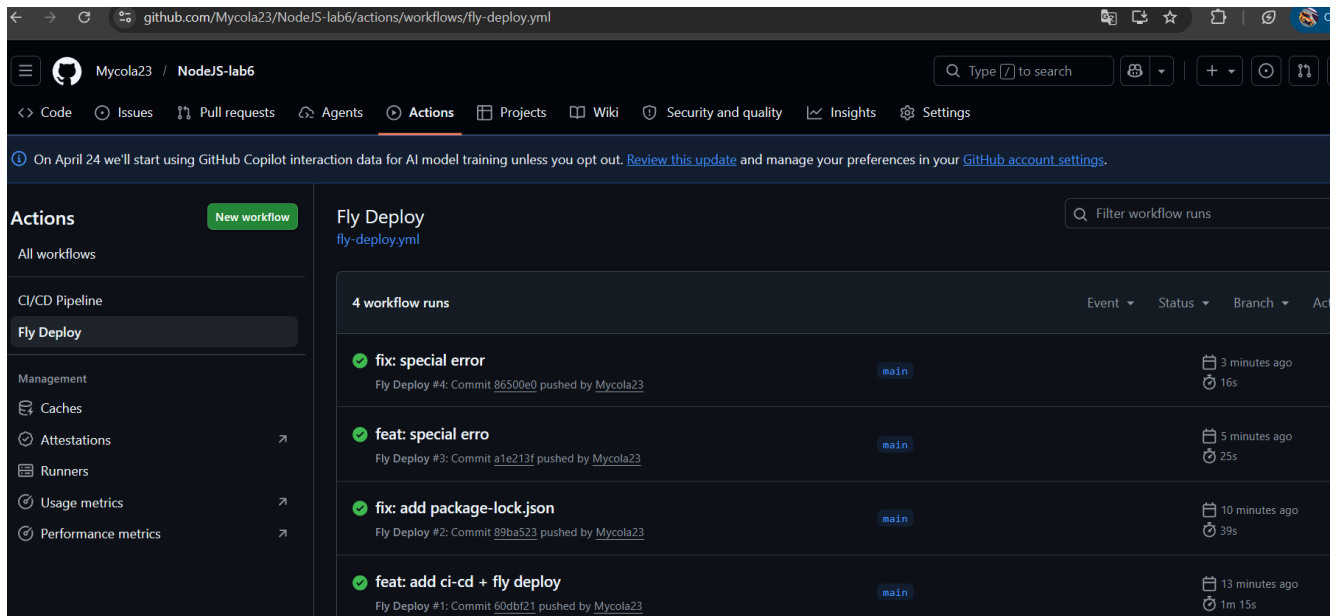


Рис.5 показ автоматичного деплою при успіху

3.4. Завдання Healthcheck у fly.toml

Налаштуйте healthcheck на боці fly.io, щоб платформа автоматично виконувала відкат при невдалому деплої.

Вимоги:

- Переконайтесь, що ендпоінт GET /health із лаб. 5 повертає 200 при активному підключенні до MongoDB та 503 при його відсутності.
- Додайте у fly.toml секцію healthcheck для HTTP-сервісу: шлях /health, метод GET, розумний час до першої перевірки після старту, регулярний інтервал та таймаут відповіді.
- Виконайте звичайний деплой через CI та у логах fly.io спостерігайте, як нова машина проходить перевірку перед тим, як прийняти трафік.
- Змоделюйте поламаний деплой: тимчасово виставте некоректне значення секрету з рядком підключення до БД (через fly secrets) та задеployте знову. Переконайтесь, що fly.io не переключаче трафік на нову машину, а попередня версія продовжує обслуговувати запити. Поверніть коректне значення секрету й переконайтесь у відновленні.

		Литвиненко М.В.			ДУ «Житомирська політехніка».26.121.01.006 – Лр6	Арк.
		Оринчак А. І.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

Додано до fly.toml

```
[[http_service.checks]]
  grace_period = "10s"
  interval = "15s"
  method = "get"
  timeout = "5s"
  path = "/health"
```

Логи коли все ок

```
2026-04-26T12:18:49Z app[48e65e5c10e318] ams [info]◇ injected env (0) from .env // tip: ⚠ override existing { override: true }
2026-04-26T12:18:51Z app[48e65e5c10e318] ams [info]Успішно підключено MongoDB
2026-04-26T12:18:51Z app[48e65e5c10e318] ams [info]server start on port 3000
2026-04-26T12:18:52Z health[48e65e5c10e318] ams [info]Health check 'servicecheck-00-http-3000' on port 3000 is now passing.
2026-04-26T12:18:52Z proxy[48e65e5c10e318] ams [info]machine became reachable in 3.390304793s
```

Логи після зміни секретів

```
PS D:\diciplens\second-course\NodeJS\lab_06> fly secrets set MONGODB_URI="mongodb+srv://wrong_user:wrong_password@cluster.mongodb.net/db"
Updating existing machines in 'lab-06' with rolling strategy

-----
✓ [1/2] Machine d899739be993e8 [app] update succeeded
✗ [2/2] Machine 48e65e5c10e318 [app] update failed: timeout reached waiting for health checks to pass for machine 48e65e5c10e318: failed to get VM 48e65e...
-----
Error: timeout reached waiting for health checks to pass for machine 48e65e5c10e318: failed to get VM 48e65e5c10e318: Get "https://api.machines.dev/v1/apps/lab-06/machines/48e65e5c10e318": net/http: request canceled
PS D:\diciplens\second-course\NodeJS\lab_06>
```

Логи тепер після правильного підключення до БД

```
PS D:\diciplens\second-course\NodeJS\lab_06> fly secrets set MONGODB_URI="mongodb+srv://wrong_user:wrong_password@cluster.mongodb.net/db"
Updating existing machines in 'lab-06' with rolling strategy

-----
✓ [1/2] Machine 48e65e5c10e318 [app] update succeeded
✓ [2/2] Machine d899739be993e8 [app] update succeeded
-----
Checking DNS configuration for lab-06.fly.dev
✓ DNS configuration verified
PS D:\diciplens\second-course\NodeJS\lab_06>
```

Посилання на GitHub репозиторій <https://github.com/Mycola23/NodeJS-lab6>

Висновки: Отже, в ході виконання лабораторної роботи було набуто практичних навичок автоматизації збірки, тестування та розгортання Node.js-застосунку у хмарі шляхом продовження роботи з контейнеризованим REST API з лабораторної роботи №5

		Литвиненко М.В.			ДУ «Житомирська політехніка».26.121.01.006 – Лр6	Арк.
		Оринчак А. І.				8
Змн.	Арк.	№ докум.	Підпис	Дата		